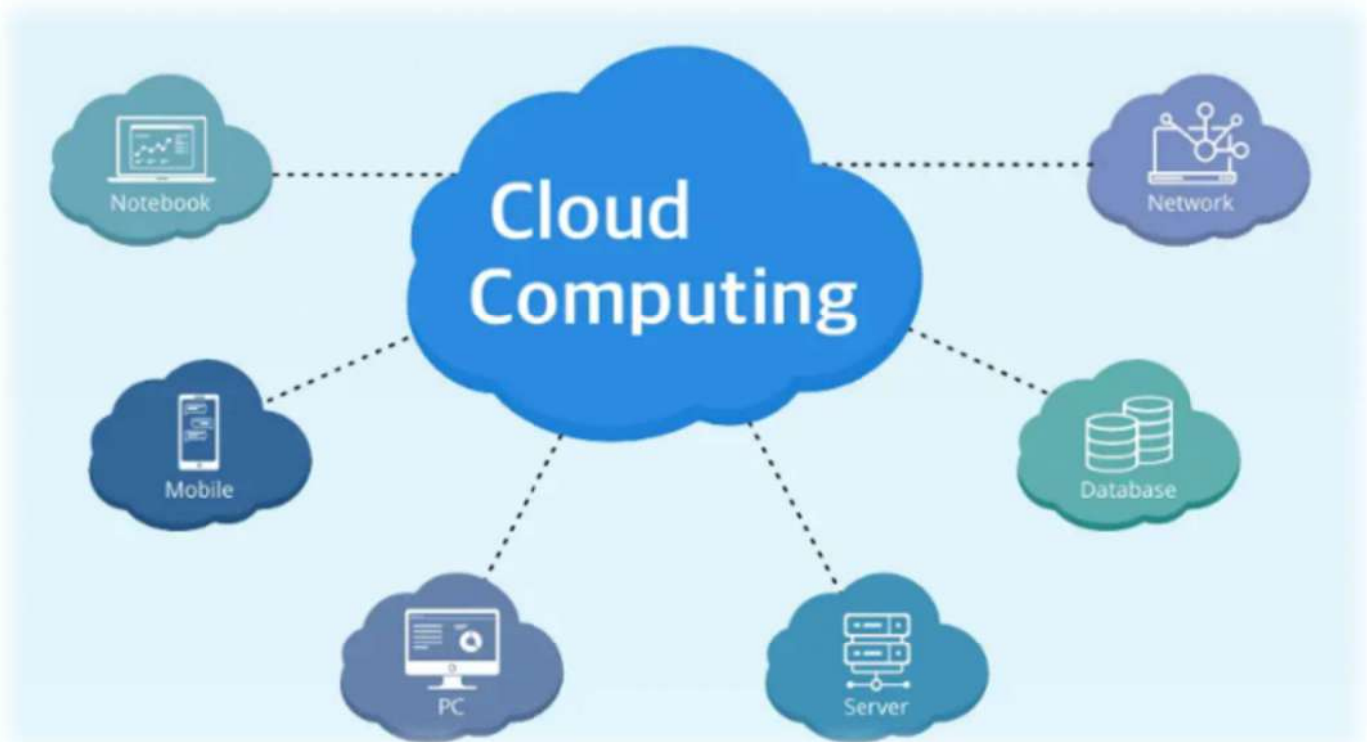


COURSE OBJECTIVE_05

Storage Systems:

- ★ Storage Models
- ★ File Systems, and Databases
- ★ Distributed File Systems
- ★ General Parallel File System
- ★ Google File Systems
- ★ Apache Hadoop
- ★ Locks and Chubby: A Locking Service
- ★ Transaction Processing and NoSQL Databases
- ★ Big Table
- ★ Mega Store



STORAGE MODELS

Cloud computing provides different types of storage systems to store, manage, and access data over the internet. Each storage type is designed for specific use cases like backups, file sharing, big data, or running virtual machines.

1. OBJECT STORAGE

Definition

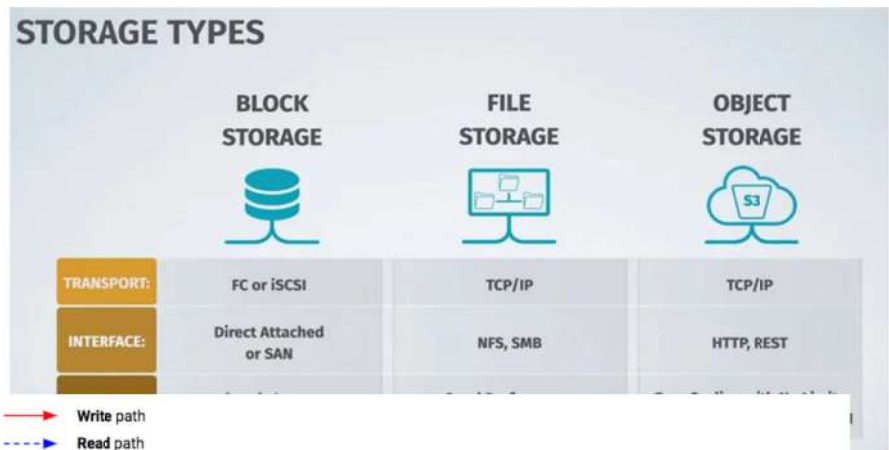
Object storage stores data as objects (file + metadata + unique ID). It is highly scalable and used for large unstructured data.

Examples

- Amazon S3
- Google Cloud Storage
- Azure Blob Storage

Use Cases

- Backups and archives
- Big data storage
- Media storage (videos, images)
- Website static content



2. BLOCK STORAGE

Definition

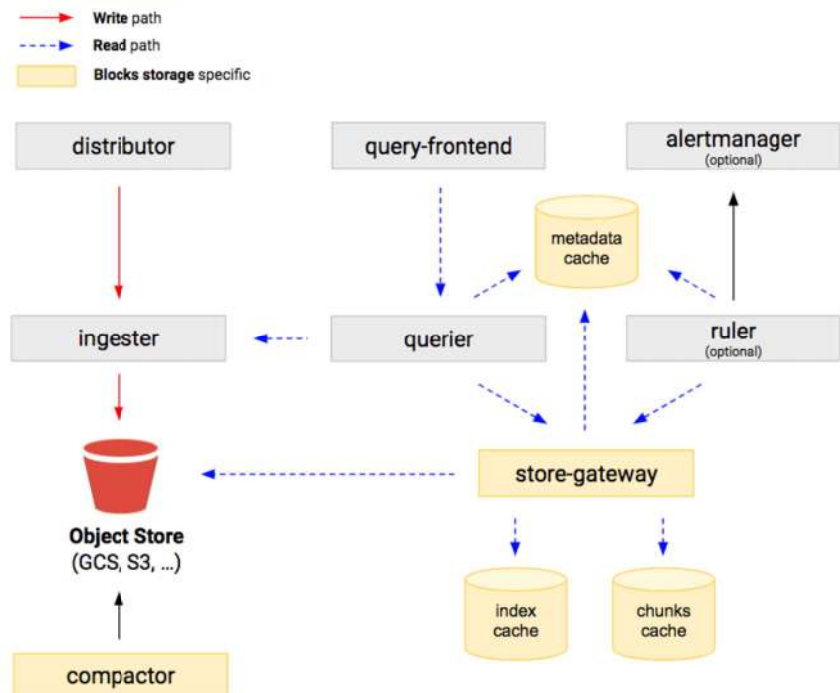
Block storage divides data into fixed-size blocks. It behaves like a virtual hard drive attached to a VM.

Examples

- AWS EBS
- Azure Disk Storage
- Google Persistent Disk

Use Cases

- Databases
- Virtual machines
- Low-latency application



3. FILE STORAGE

Definition

File storage uses a shared file system accessible via protocols like NFS, SMB.

Examples

- AWS EFS
- Azure Files
- Google Filestore

Use Cases

- Shared directories
- User home folders
- Content management systems

4.LOCAL / INSTANCE STORAGE

Definition

Storage physically attached to the cloud server (temporary).

Characteristics

- Very fast
- Data lost when instance stops

Use Cases

- Caching
- Temporary data processing
- Buffer storage

5.BACKUP STORAGE

Definition

Used to store backup copies of data, snapshots, and disaster recovery files.

Examples

- AWS Backup
- Azure Backup

Use Cases

- Disaster Recovery (DR)
- Snapshots
- Long-term retention

6.ARCHIVAL / COLD STORAGE

Definition

Low-cost storage for data that is rarely accessed.

Examples

- Amazon S3 Glacier
- Azure Cool/Archive tier

Use Cases

- Compliance records
- Old logs
- Medical records
- Historical data

7.STORAGE ACCESS METHODS

Method	Meaning	Used In
NFS (Network File System) / SMB (Server Message Block)	File access over network	File storage
ISCSI (Internet Small Computer System Interface)	Block storage over IP(Internet Protocol)	Block storage
REST API (Representational State Transfer Application Programming Interface)	Access objects using URLs(Uniform Resource Locator)	Object storage

8. KEY FEATURES OF CLOUD STORAGE SYSTEMS

- Scalability (grow without limit)
- Durability (99.99999999% for S3)
- High availability

- Pay-as-you-go pricing
- Security & encryption
- Replication across regions

FILE SYSTEMS AND DATABASES

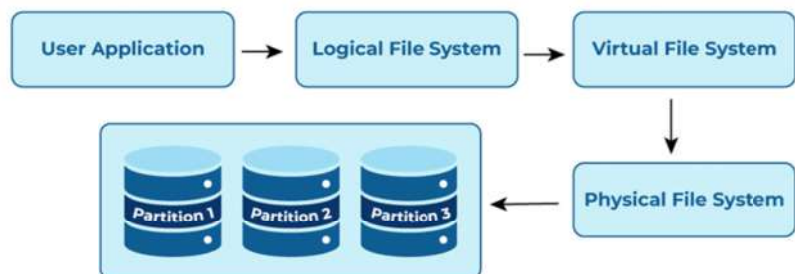
What is a File System?

A file system (FS) controls how data is stored, organised, retrieved, and managed on storage devices.

Key Responsibilities

- File naming & organisation
- Space allocation & directory structure
- Access control & security
- Crash recovery
- Metadata management

The Architecture of a File System



File System Architecture

A typical OS file system includes:
Layers

1. Application layer – user programs
2. Logical file system – manages metadata, directory structure
3. File organisation module – maps logical blocks → physical blocks
4. Basic file system – communicates with device drivers
5. I/O control – device drivers & interrupt handlers
6. Devices – storage hardware

File systems and databases are both methods for storing and managing digital data, but they operate at different levels of abstraction and offer distinct features. A **file system manages unstructured data** as individual files in a hierarchy, while a **database manages structured, related data** using a dedicated software system called a DBMS.

File System

A file system is an operating system subsystem that manages how data is stored and retrieved on physical storage devices like hard drives and flash drives.

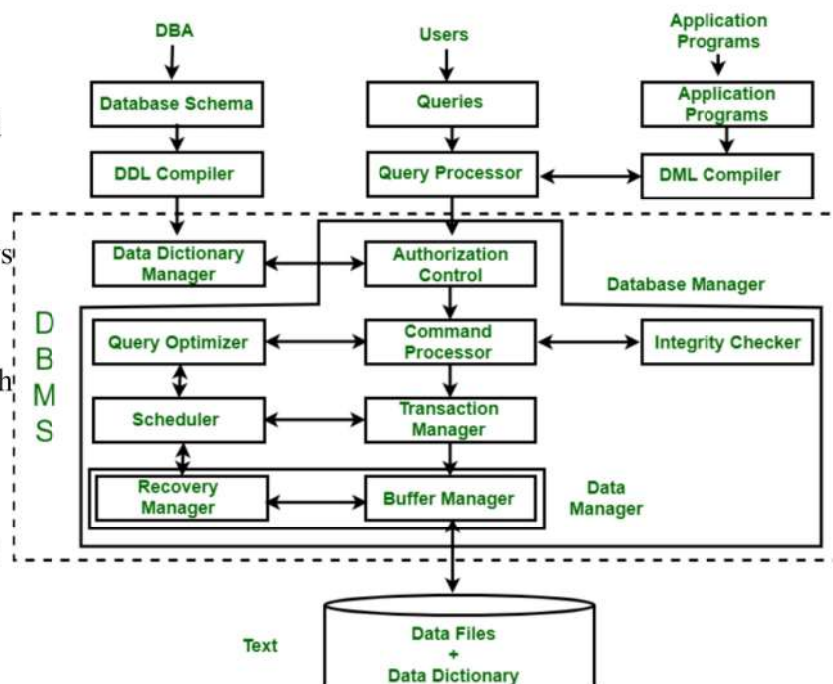
- **Structure:** Data is organised hierarchically into directories (folders) and subdirectories containing individual files.
- **Data Type:** It primarily handles unstructured data of arbitrary types, such as documents, images, audio files, and logs.
- **Access:** Access is generally direct, with operations like creating, reading, writing, renaming, and deleting files managed by the operating system.
- **Querying:** File systems offer limited search capabilities, typically based on file names, locations, or timestamps. Complex searches based on the content of files are not built-in.

What are Data Bases?

Data Bases:

A Database Management System (DBMS) is specialised software that provides an efficient and organised environment for storing, managing, and retrieving large volumes of structured, interrelated data. Databases are often built *ON TOP* of file systems.

- **Structure:** Data is organised into structured formats, typically tables (in relational databases) with defined rows and columns, allowing relationships to be established between different data sets using keys and constraints.
- **Data Type:** It specialises in managing structured information with well-defined formats.
- **Access:** Data is accessed and manipulated using a specialised query language, such as SQL, which allows for complex operations.
- **Querying:** Databases are optimised for fast and flexible data retrieval, supporting complex queries, sorting, filtering, and aggregation through indexing and optimisation techniques.



Key Differences

Feature	File System	DBMS
Data Organisation	Hierarchical files and directories	Structured tables with defined relationships
Data Redundancy	High, as data can be repeated across files	Low, reduced through normalisation processes
Data Integrity	Low; integrity checks depend on individual programs	High; enforced via built-in constraints and rules
Query Capability	Limited to basic file attributes	Strong; supports complex queries (e.g., SQL)
Concurrency	Limited support for multiple simultaneous users	Strong support with mechanisms like locking/transactions
Security	Basic, limited to file-level permissions	Robust, with user authentication and fine-

grained access control

Backup & Recovery

Manual, time-consuming process

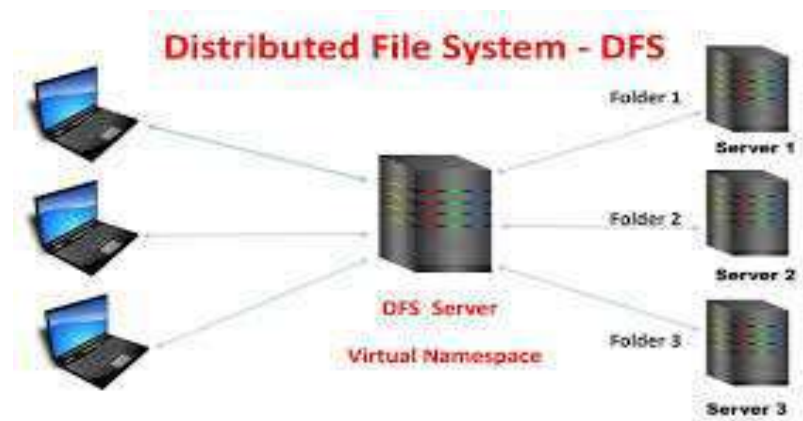
Automated, with built-in crash recovery mechanisms

DISTRIBUTED FILE SYSTEMS

A **Distributed File System (DFS)** is a system that manages file storage across multiple physical servers in a network, allowing users and applications to access the data as if it were stored on a single, local machine. This provides scalability, high availability, and data redundancy, making it essential for modern computing environments like cloud computing and big data analytics.

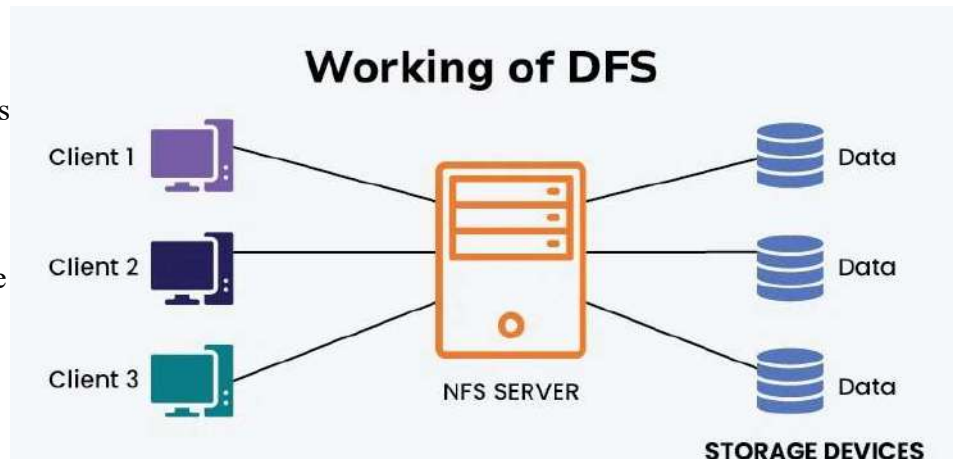
Key Features and Principles

- **Transparency:** Users don't need to know the physical location of files or the underlying architecture of the system. This includes:
 - **Access Transparency:** Local and remote files are accessed using the same interface and methods.
 - **Location Transparency:** The file's name does not indicate its physical location, so files can move without the name changing.
 - **Replication Transparency:** If files are replicated, the user is unaware of the multiple copies or their locations.
- **Scalability:** The system can grow by adding more nodes (servers) to accommodate increasing data and user demands without significant performance degradation.
- **Fault Tolerance and High Availability:** By replicating data across multiple nodes, the system can continue to operate even if one or more servers fail.
- **Concurrency:** Multiple users can access and modify files simultaneously, with mechanisms like file locking or versioning to manage data consistency and integrity.
- **Performance:** Techniques such as client-side caching, load balancing, and data locality (processing data where it is stored) are used to minimise latency and optimise performance.



Key Components

- **Namespace:** Provides a unified, logical view of the distributed files and folders, abstracting their physical locations.
- **Metadata Management:** A dedicated service or set of servers that stores and manages information about files, such as names, sizes, permissions, and their physical locations (mapping logical names to data blocks).
- **Data Storage Nodes:** The servers or machines that store the actual file data, typically in fixed-size units called blocks or chunks.
- **Replication Mechanisms:** Protocols and algorithms that create and synchronise multiple copies (replicas) of data across different nodes to ensure availability and reliability.



Common Implementations

- **Network File System (NFS):** A widely used client-server protocol, primarily in Unix and Linux environments, for remote file access.
- **Server Message Block (SMB) / Common Internet File System (CIFS):** A protocol used predominantly in Windows environments for file and printer sharing across a network. Microsoft's DFS implementation uses SMB.
- **Hadoop Distributed File System (HDFS):** An open-source, fault-tolerant file system designed for the large data sets and high-throughput processing needs of big data applications using commodity hardware.
- **Google File System (GFS):** A proprietary system developed by Google for its large-scale data processing needs, optimised for large files and high aggregate throughput.
- **CephFS and GlusterFS:** Open-source distributed file systems known for their scalability and use in cloud and high-performance computing (HPC) environments.

Uses and Applications

Distributed file systems are crucial for scenarios requiring massive storage capacity and reliable data access across multiple locations. Common applications include:

- **Big Data Analytics:** HDFS is a core component of the Apache Hadoop ecosystem for storing and processing petabytes of data.
- **Cloud Storage Services:** Providers like Amazon S3 and Google Cloud Storage use distributed systems to offer scalable and durable storage.
- **Enterprise Collaboration:** Allows geographically dispersed teams to access and share files seamlessly and work on the latest versions in real-time.
- **High-Performance Computing (HPC):** Systems like Lustre and IBM GPFS are designed for the demanding I/O workloads of scientific computing and supercomputing.

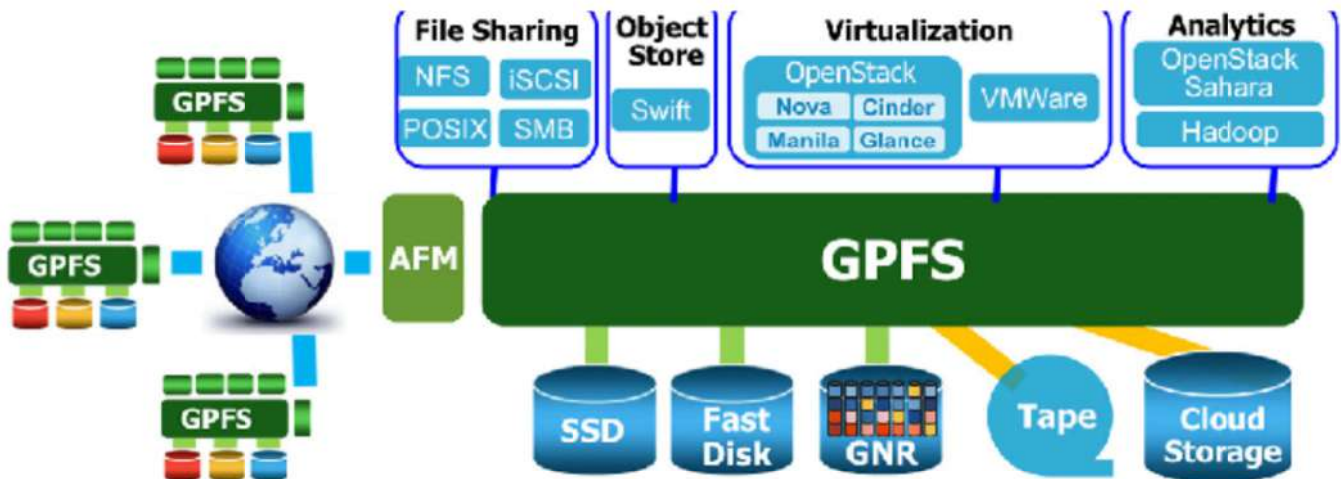
GENERAL PARALLEL FILE SYSTEM

A **general parallel file system (GPFS)** in cloud computing is a high-performance, distributed storage architecture that allows multiple compute nodes to access shared data concurrently across a network. This approach distributes data across several storage servers and uses parallel I/O paths to achieve high throughput and low latency, making it ideal for data-intensive workloads in High-Performance Computing (HPC) environments, AI/ML, and big data analytics.

Key Concepts & Features

- **Parallel Access:** Multiple clients can read from and write to the same file simultaneously, which significantly improves data throughput and reduces access times compared to traditional network file systems (NFS).
- **Data Striping:** Files are broken into smaller blocks (stripes) and distributed across multiple storage nodes or drives. This allows for concurrent data transfers from multiple locations, enhancing performance.
- **Scalability:** PFS can scale out by adding more storage nodes, accommodating petabytes or exabytes of data and thousands of clients without compromising performance.
- **Distributed Metadata:** Metadata (information about files, their locations, etc.) is also distributed across the system or managed by dedicated metadata servers to avoid single-point-of-failure and performance bottlenecks.

- **High Availability and Fault Tolerance:** Features like data replication (e.g., storing three copies of a data chunk) and redundancy mechanisms ensure continuous data access even if some nodes fail.
- **POSIX Compliance:** Many systems, like IBM Spectrum Scale, offer full POSIX compatibility, allowing existing applications to run without modification.



Popular General Parallel File Systems Used in the Cloud

Several parallel file systems are used in cloud environments, often available as managed services from major cloud providers:

- **IBM Spectrum Scale (formerly GPFS):** A robust, high-performance clustered file system widely used in enterprise and supercomputing centers. It is available in the cloud through partners and managed solutions like the Sycomp Intelligent Data Storage Platform on Google Cloud.
- **Lustre:** An open-source parallel file system renowned for its exceptional performance in large-scale cluster computing environments. Google offers Google Cloud Managed Lustre as a service for high-performance workloads.
- **BeeGFS:** Known for its performance, flexibility, and ease of installation and management, it is a popular choice in academic and research institutions and is also used in cloud setups.
- **CephFS:** An open-source, unified storage platform that provides file, block, and object storage interfaces, suitable for large-scale cloud and big data applications.

These systems help bridge the gap between traditional on-premises HPC infrastructure and scalable, on-demand cloud resources, enabling complex data-intensive applications to run efficiently in the cloud.

GOOGLE FILE SYSTEM (GFS)

The Google File System (GFS) is a distributed, scalable file system developed by Google to store huge files (GB–TB) across thousands of commodity servers.

It is optimised for:

- Large sequential reads/writes
- Append-heavy workloads (logs, indexing)
- High throughput, not low latency
- Fault tolerance

COMPONENTS OF GFS

1) GFS Master

Stores all metadata:

- File names & directory tree
- Mapping of file → chunks
- Chunk locations
- Access control
- Garbage collection
- Chunk creation, re-replication

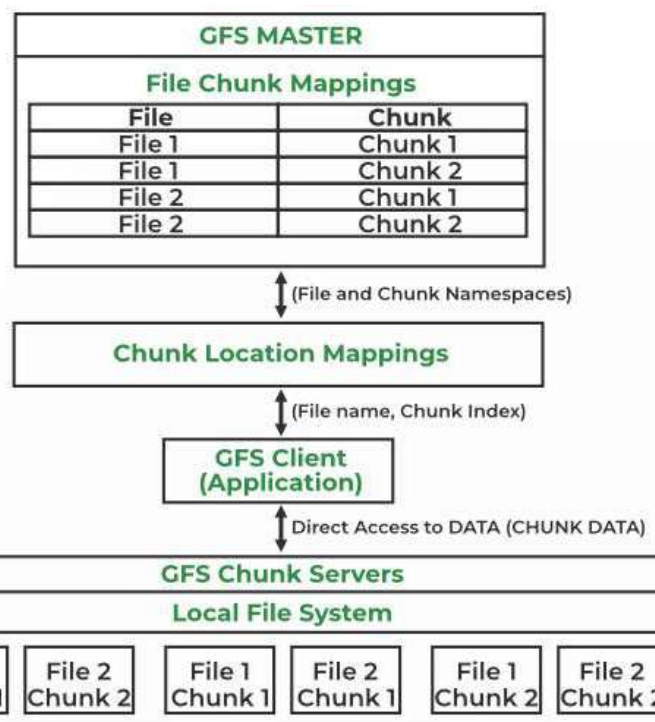
💡 Master does not store file data, only metadata.

2) Chunkservers

- Store file data in 64 MB chunks
- Each chunk is replicated (usually 3 copies)
- Chunkservers send periodic heartbeats to the master
- Chunk 1 → 64 MB
- Chunk 2 → 64 MB
- Chunk 3 → 64 MB
- Each chunk is identified by a 64-bit chunk handle.

3) GFS Clients

- Request metadata from Master
- Read/write data directly from Chunkservers (not through master)
- Reduces bottlenecks



⚙️ HOW GFS WORKS

★ 1. READ Operation

- Client → Master : Ask for chunk location
- Master → Client : List of chunk servers
- Client → Chunk-server : Reads data directly

Steps:

- Client asks master for the chunk location
- Master responds with replica locations
- Client contacts nearest chunk-server and reads data

★ 2. WRITE Operation

- Client → Master : Request primary replica
- Master → Client : Primary + secondary replicas
- Client → Replicas : Push data
- Primary → Secondaries : Apply mutation

Steps:

- Client requests master to identify primary replica
- Master assigns 1 primary + 2 secondary replicas
- Client forwards data to all replicas
- Primary applies the write and instructs secondaries
- All replicas acknowledge success

🛡️ FAULT TOLERANCE IN GFS

- **Replication (default = 3 replicas)**
If one chunkserver fails, replicas take over.
- **Automatic Re-Replication**
Master creates new replicas if one is lost.
- **Master Shadow Servers**
Backup masters for recovery.
- **Heartbeat Monitoring**
Chunkserver failures are detected quickly.

🏠 ADVANTAGES OF GFS

- Highly scalable (thousands of machines)
- High throughput for large data processing
- Fault-tolerant (via replication)
- Optimized for large sequential workloads
- Uses cheap commodity hardware
- Suitable for MapReduce, indexing, logging

△ LIMITATIONS OF GFS

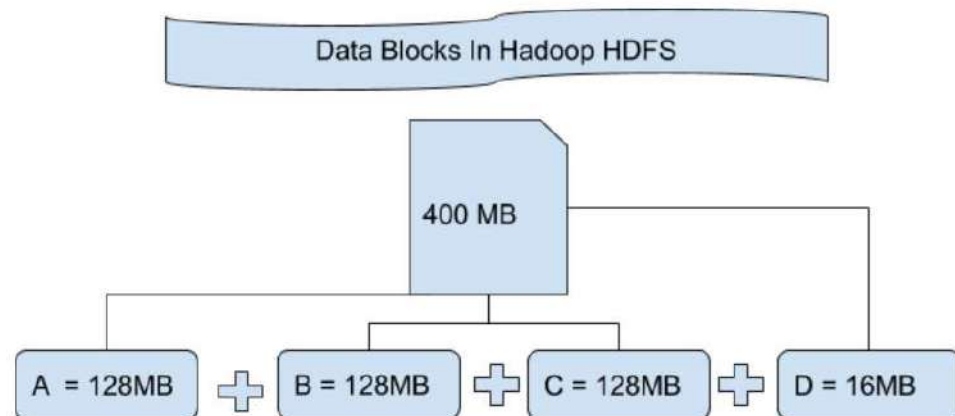
- Single master → potential bottleneck
- Not suitable for small files
- High latency for random writes
- Designed for Google → not general-purpose

APACHE HADOOP

Apache Hadoop is an **open-source framework** used for **distributed storage** and **parallel processing** of large datasets (Big Data) across clusters of commodity hardware.

It provides:

- **Scalable** data storage
- **Fault-tolerant** computing
- **Parallel processing**
- A complete **ecosystem** for handling big data



Apache Hadoop is a foundational framework for big data that is widely used in cloud computing environments to provide scalable, cost-effective, and fault-tolerant storage and processing of massive datasets. Major cloud providers offer managed services that simplify running Hadoop clusters.

Role of Hadoop in the Cloud

In cloud computing, Hadoop is primarily used to manage and process big data without the need for expensive, high-end, on-premises hardware. The cloud infrastructure, with its inherent scalability and flexibility, provides an ideal platform for Hadoop's distributed architecture.

Key benefits of using Hadoop in the cloud:

- **Scalability:** Cloud platforms allow users to easily scale compute and storage resources up or down as needed, which is ideal for the varying workloads of big data processing.

- **Cost-Effectiveness:** By leveraging the cloud's commodity hardware and "pay-as-you-go" pricing models, organizations can significantly reduce infrastructure costs compared to managing their own data centers.
- **Fault Tolerance and High Availability:** Hadoop is designed to handle hardware failures at the application layer by replicating data across multiple nodes. This ensures data availability even if a node fails, a feature enhanced by the distributed nature of cloud storage like Amazon S3 or Azure Blob Storage.
- **Flexibility:** Hadoop in the cloud can store and process various data formats (structured, semi-structured, and unstructured) in a central repository, often referred to as a data lake.

Core Hadoop Components in a Cloud Context

The main components of the Apache Hadoop framework are essential for its operation in the cloud:

- **Hadoop Distributed File System (HDFS):** This is Hadoop's primary storage layer, designed for high-throughput access to large files. In the cloud, HDFS can run on virtual machines, or it can be integrated with cloud-native object storage services like Amazon S3, Azure Data Lake Storage Gen2, or Google Cloud Storage through connectors like EMRFS.
- **Yet Another Resource Negotiator (YARN):** YARN is the framework for managing compute resources within a cluster and scheduling user applications. It allows various processing engines (like Spark, Hive, or MapReduce) to run and share resources efficiently on the same cluster.
- **MapReduce:** This is the original programming model for parallel processing of large datasets. It breaks down tasks into smaller "map" tasks and aggregates the results in "reduce" tasks, all running in parallel across the cluster nodes.

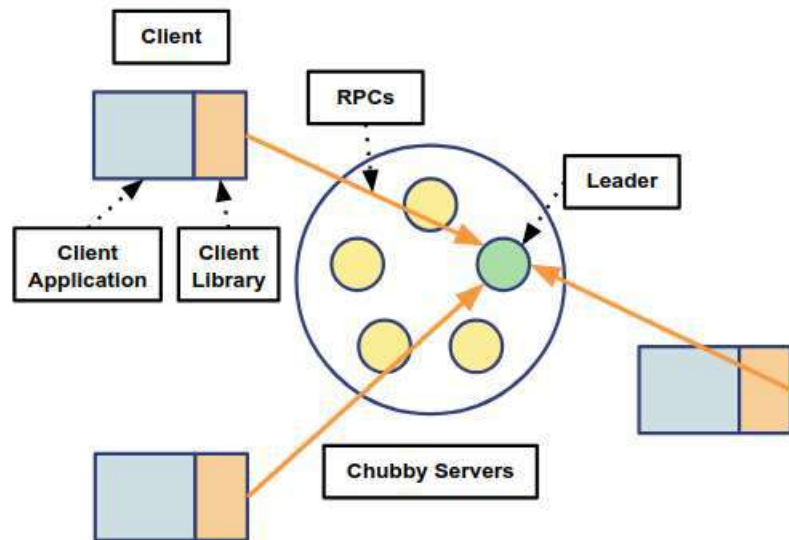
Major Cloud Provider Solutions

- Leading cloud service providers offer managed big data platforms that use Apache Hadoop and its ecosystem tools:
- **Amazon Web Services (AWS):** Offers Amazon EMR (Elastic MapReduce), a managed cluster platform that simplifies running Hadoop, Spark, and other big data frameworks.
- **Microsoft Azure:** Provides Azure HDInsight, a fully managed, open-source analytics service that includes support for Apache Hadoop clusters.

- Google Cloud (GCP): Offers Dataproc, a fast, easy-to-use, fully managed service for running Apache Hadoop and Apache Spark clusters, integrated with other GCP services

LOCKS AND CHUBBY: A LOCKING SERVICE

Locks" in cloud computing generally refers to **distributed locks**, a mechanism for coordinating access to shared resources across multiple independent machines to maintain data consistency. **Chubby** is Google's specific, highly influential distributed lock service, designed for reliable, coarse-grained synchronisation in large-scale systems like the Google File System (GFS) and Big-table.



Distributed Locks

In a cloud (distributed) environment, traditional, single-server locking mechanisms are insufficient because multiple clients across different machines may need to access the same resource simultaneously. Distributed locking ensures that only one process can act on a shared resource at a time, preventing conflicts and data corruption.

Key characteristics and uses:

- **Mutual Exclusion:** Enforces that critical sections of code are not executed concurrently by multiple processes.
- **Consistency:** Helps maintain data integrity and consistency across distributed databases and services.
- **Leader Election:** A common use case where multiple potential primary servers compete for a lock; the winner becomes the master or leader, and others become replicas.

Google Chubby

Chubby was developed by Google to provide a simple, file system-like interface for coarse-grained locking and reliable, low-volume storage.

- **Design Philosophy:** Chubby priorities **availability and reliability** over high performance or high-volume data storage. Locks are typically held for long durations (hours or days) rather than seconds.

- **Architecture:** A Chubby "cell" consists of a small set of servers (typically five replicas) that use the Paxos consensus protocol to elect a master. Clients communicate with the master, which manages locks and data consistency.
- **Advisory Locks:** Locks are "advisory," meaning clients are expected to check for a lock before accessing a resource; the system doesn't automatically prevent access. This makes it easier for developers to integrate into existing systems.
- **Small File Storage:** Acts as a hierarchical name service (like a file system) for storing small amounts of metadata and configuration data (e.g., the elected master's network address).
- **Event Notification:** Clients can subscribe to events (like file changes or master failover) and receive asynchronous notifications, avoiding constant polling.
- **Client Caching:** Clients maintain a consistent, write-through cache of data to reduce network traffic to the master. The master sends invalidations when data changes.
- **Sessions and KeepAlives:** Clients maintain a session via periodic "KeepAlive" messages. If a session expires (due to client or network failure), all locks held by that client are automatically released, allowing other clients to proceed.

In summary, Chubby is a foundational distributed coordination service that made it simpler for developers to build large, reliable distributed systems by abstracting complex consensus logic behind a familiar locking mechanism. Apache ZooKeeper is a well-known open-source equivalent inspired by Chubby.

TRANSACTION PROCESSING AND NOSQL DATABASES

In cloud computing, traditional transactional processing is typically associated with relational (SQL) databases, which prioritize strong consistency and rigid schemas. NoSQL databases, in contrast, were developed for flexibility, massive scale, and high availability, often relaxing traditional transactional properties (ACID) in favor of the BASE (Basically Available, Soft state, Eventual consistency) model. However, the landscape has evolved, and modern cloud NoSQL services increasingly offer strong transactional consistency.

Transaction Processing

Transaction processing ensures data integrity through **ACID properties**:

- **Atomicity:** A transaction is all or nothing; it either completes entirely or fails completely.
- **Consistency:** The database remains in a valid state before and after the transaction.
- **Isolation:** Concurrent transactions do not interfere with each other, producing the same result as if they ran sequentially.
- **Durability:** Once a transaction is committed, the changes are permanent, even in the event of a system failure.

Traditional relational databases are built around providing strict ACID compliance.

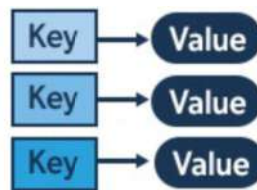
NoSQL Databases in Cloud Computing

NoSQL databases are non-relational, use flexible schemas, and are designed for horizontal scaling across distributed server clusters in the cloud. They are categorized by their data models:

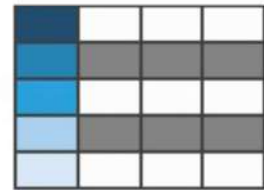
- **Key-value stores:** Simple but fast access (e.g., Redis, Amazon DynamoDB).
- **Document databases:** Store data in flexible formats like JSON (e.g., MongoDB, Google Cloud Datastore).
- **Wide-column stores:** Optimize for read/write performance across large datasets (e.g., Cassandra).
- **Graph databases:** Designed to manage data with complex interconnections (e.g., Amazon Neptune, Neo4j).

NoSQL

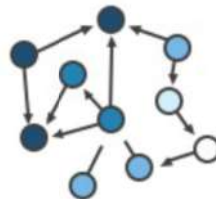
Key-Value



Column-Family



Graph



Document



The Intersection of Transactions and NoSQL in the Cloud

While early NoSQL databases often compromised on immediate consistency for performance and scale (eventual consistency), many modern cloud-native NoSQL services now support robust transactional capabilities:

- **Selective ACID Support:** Many NoSQL databases can now be configured to support full ACID transactions, particularly within a single document or across multiple documents, depending on the use case and configuration (e.g., MongoDB and Oracle NoSQL Database Cloud Service offer ACID transactions).
- **Optimized Data Modeling:** NoSQL databases often store related data within a single structure (like a JSON document), which can eliminate the need for multi-record or multi-table transactions required in relational models, simplifying transaction management.
- **Cloud Provider Offerings:** Major cloud providers offer fully managed NoSQL services with built-in high availability and data replication, which handle the underlying complexity of ensuring durability and consistency at a massive scale:
 - Amazon DynamoDB
 - Microsoft Azure Cosmos DB
 - Google Cloud Datastore
 - Oracle NoSQL Database Cloud Service

In summary, NoSQL databases in the cloud are excellent for high-volume . Cloud Bigtable is a fully managed, high-performance, NoSQL database service offered by Google Cloud Platform (GCP). It is designed to handle massive amounts of structured and semi-structured data, scaling to petabytes of data across potentially billions of rows and thousands of columns.

Key Features and Data Model

- **NoSQL Wide-Column Store:** Bigtable is fundamentally a key-value store, often referred to as a wide-column database. Data is organized into tables as a distributed, multi-dimensional sorted map indexed by a row key, column key, and timestamp (row:string, column:string, Timestamp:int64) → string.
- **Massive Scalability:** It offers seamless horizontal scalability, allowing you to increase throughput by simply adding or removing cluster nodes without downtime. It can handle millions of reads and writes per second.
- **High Performance & Low Latency:** Optimized for high throughput and single-digit millisecond latency, making it ideal for real-time applications and demanding workloads.
- **Fully Managed:** Google manages the underlying infrastructure, including hardware provisioning, software updates, and maintenance, allowing developers to focus on application development.
- **Durability and Availability:** Data is stored on Google's highly durable file system, Colossus, and is automatically replicated across multiple zones or regions for high availability and fault tolerance.
- **Integration with the Apache Ecosystem:** Bigtable supports the Apache HBase API, simplifying migration and integration with existing big data tools like Hadoop, Spark, and Dataflow.
- **Data Model Flexibility:** It supports a flexible schema, where columns not used in a specific row take up no space, making it efficient for sparse datasets.

Common Use Cases

Bigtable is an ideal choice for applications requiring massive scale and high-speed data access:

- **Time-series data:** Storing metrics like CPU and memory usage over time for numerous servers.
- **IoT (Internet of Things):** Ingesting and analyzing large volumes of sensor data in real-time.
- **Financial Data Analysis:** Managing stock prices, transaction histories, and detecting fraud patterns.

- **Machine Learning:** Serving as a storage engine for batch MapReduce operations and machine-learning applications.
- **AdTech and Social Media:** Storing user profiles, posts, ad impressions, and interaction data that requires high throughput.

Integration with other Google Cloud services

Bigtable integrates seamlessly with other GCP services to build complete data processing pipelines:

- **BigQuery:** For running complex, ad-hoc SQL-based analytics on data stored in Bigtable.
- **Dataflow/Dataproc:** For processing data using Apache Beam, Hadoop, or Spark.
- **Vertex AI Vector Search:** To store vector embeddings for building recommendation engines and AI applications.

BIGTABLE

Google Bigtable is a fully managed, massively scalable NoSQL database service designed for large analytical and operational workloads. It is optimized for high read and write throughput at low latency and is used internally by many core Google services like Search, Maps, and YouTube.

Key Features

NoSQL Wide-Column Store: Bigtable uses a sparsely populated, wide-column data model organized as a sorted key-value map.

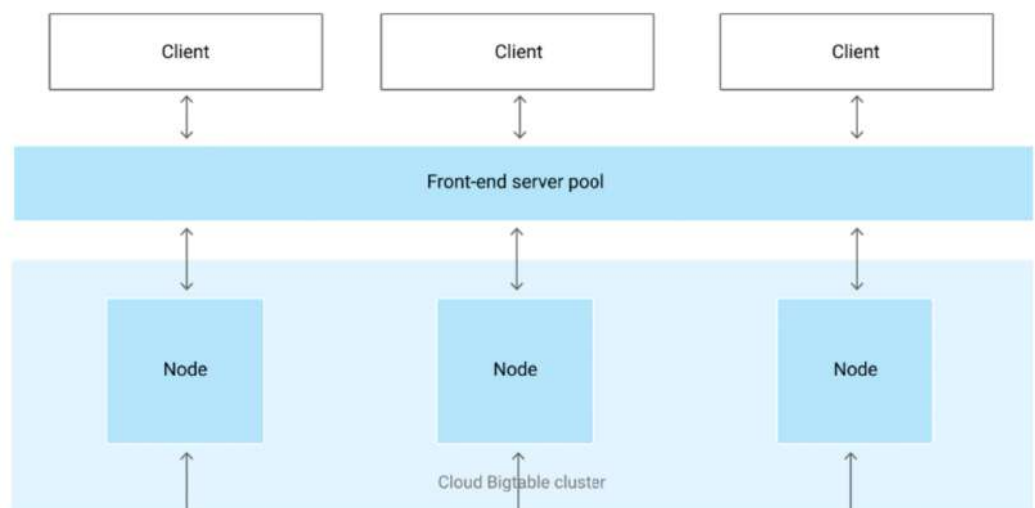
Data is indexed by a single row key, allowing for efficient access to related information stored in contiguous rows.

Massive Scalability: The service is designed to scale to petabytes of data across thousands of servers. It decouples compute and storage, allowing you to seamlessly add or remove

cluster nodes without downtime to handle fluctuating workloads.

High Performance: Bigtable offers single-digit millisecond latency and high throughput, making it ideal for real-time applications. Performance scales linearly as nodes are added to a cluster.

Data Model Flexibility: It can store various data types, including scalars, JSON, and vector embeddings, and allows the data model to evolve organically by dynamically adding or removing columns as needed.



Open-Source Compatibility: Bigtable provides Apache HBase and Cassandra-compatible APIs and integrates with the broader Apache ecosystem, including Hadoop, Spark, and Dataflow, for easier migration and use.

SQL Support: In addition to its NoSQL APIs, Bigtable supports SQL queries using GoogleSQL for low-latency application development and Spark SQL for data science use cases.

Common Use Cases

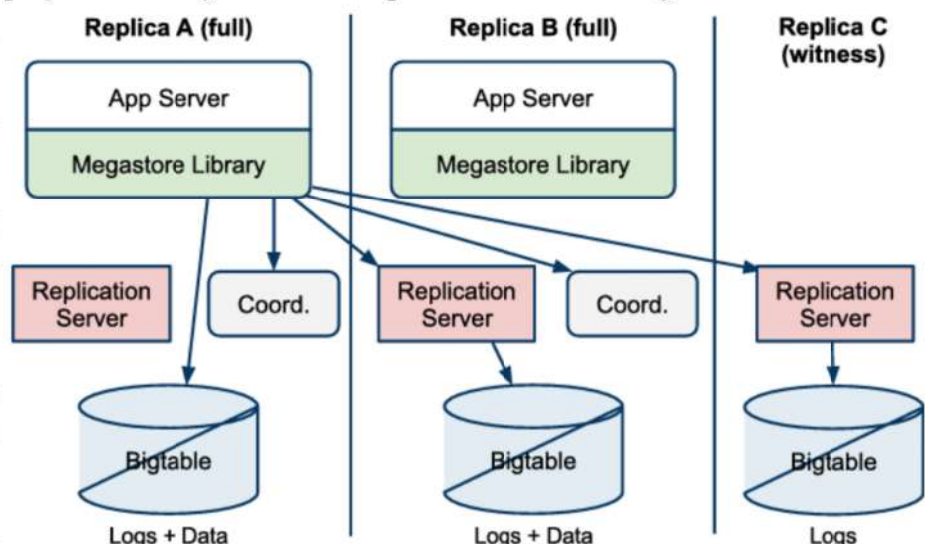
- **Bigtable:** It is well-suited for applications requiring massive scale and high-speed data access.
- **Time-series data:** Storing metric data like CPU and memory usage from multiple servers.
- **IoT (Internet of Things) data:** Managing large volumes of usage reports from smart home appliances and sensors.
- **Financial data:** Handling transaction histories, stock prices, and currency exchange rates.
- **Machine learning:** Serving as a storage engine for training ML models and performing similarity searches over vector embeddings.
- **Operational analytics:** Supporting real-time analytics and monitoring in user-facing applications.

MEGASTORE

The term "Megastore" in cloud computing refers to a specific, highly scalable and available **storage system developed by Google**. It was designed as a middle ground between traditional relational database management systems (RDBMS) and NoSQL solutions, offering both scalability and strong consistency guarantees.

Key Features of Google's Megastore

- **Hybrid Data Model:** Megastore blends the structured data model and ACID transaction properties (atomicity, consistency, isolation, durability) of an RDBMS with the immense scalability of a NoSQL datastore like Google's Bigtable, which it is built upon.
- **Scalability and High Availability:** The system partitions data into "entity groups" and replicates each group synchronously across multiple data centers using the Paxos consensus algorithm. This approach ensures high availability and fault tolerance, even in the event of a datacenter failure.
- **Consistency:** It provides fully serializable ACID semantics within a



single entity group. For operations spanning multiple entity groups, it uses an asynchronous messaging system, offering a balance between strong consistency and performance.

- **Performance:** Megastore is optimized for high-volume, interactive online services. It handles billions of reads and writes daily and is designed for predictable performance, prioritizing fast reads by allowing them from any up-to-date local replica.
- **Usage:** It has been used internally by Google for numerous production applications, including the original Google App Engine Datastore, which has since evolved into services like the modern Google Cloud Datastore and Cloud Spanner.